

vLLM专题（六）-Pooling模型

 大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续费

 您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 >

vLLM 还支持池化模型，包括嵌入、重排序和奖励模型。

在 vLLM 中，池化模型实现了 VllmModelForPooling 接口。这些模型使用 Pooler 提取输入的最终隐藏状态，然后返回结果。

注意

我们目前支持池化模型主要是出于方便考虑。如兼容性矩阵所示，大多数 vLLM 功能不适用于池化模型，因为它们仅在生成或解码阶段工作，因此性能提升可能不会很大。

对于池化模型，我们支持以下 `--task` 选项。选择的选项将设置默认的池化器，用于提取最终的隐藏状态：

Task	Pooling Type	Normalization	Softmax
Embedding (<code>embed</code>)	<code>LAST</code>		X
Classification (<code>classify</code>)	<code>LAST</code>	X	
Sentence Pair Scoring (<code>score</code>)	*	*	*
Reward Modeling (<code>reward</code>)	<code>ALL</code>	X	X

- 默认的池化器始终由模型定义。

注意

如果模型在 vLLM 中定义了自己的池化器，则默认池化器将设置为该池化器，而不是表中指定的池化器。

在加载 Sentence Transformers 模型时，我们会根据其 Sentence Transformers 配置文件（modules.json）尝试覆盖默认的池化器。

提示

您可以通过 `--override-pooler-config` 选项自定义模型的池化方法，该选项优先于模型和 Sentence Transformers 的默认设置。

1. 离线推理

LLM 类提供了用于离线推理的各种方法。请参阅引擎参数以查看初始化模型时的选项列表。

1.1 LLM.encode

`encode` 方法对 vLLM 中的所有池化模型可用。它直接返回提取的隐藏状态，这对于奖励模型非常有用。

```
llm = LLM(model="Qwen/Qwen2.5-Math-RM-72B", task="reward")
(output,) = llm.encode("Hello, my name is")

data = output.outputs.data
print(f>Data: {
    data!r})
```

1.2 LLM.embed

`embed` 方法为每个提示输出一个嵌入向量。它主要为嵌入模型设计。

```
llm = LLM(model="intfloat/e5-mistral-7b-instruct", task="embed")
(output,) = llm.embed("Hello, my name is")

embeds = output.outputs.embedding
print(f"Embeddings: {
    embeds!r} (size={
    len(embeds)})")
```

可以在这里找到代码示例：[examples/offline_inference/embedding.py](#)

1.3 LLM.classify

`classify` 方法为每个输入返回一个概率向量。它主要用于分类模型。

```
llm = LLM(model="jason9693/Qwen2.5-1.5B-apeach", task="classify")
(output,) = llm.classify("Hello, my name is")

probs = output.outputs.probs
print(f"Class Probabilities: {
    probs!r} (size={
    len(probs)})")
```

代码示例可以在这里找到：[examples/offline_inference/classification.py](#)

1.4 LLM.score

方法输出句子对之间的相似度分数。它主要用于跨编码器模型。这些类型的模型在 RAG 系统中作为候选查询-文档对的重排序器。

注意

vLLM 仅能执行 RAG 的模型推理组件（例如嵌入、重排序）。要在更高层次处理 RAG，您应使用像 LangChain 这样的集成框架。

```
llm = LLM(model="BAAI/bge-reranker-v2-m3", task="score")
(output,) = llm.score("What is the capital of France?",
                      "The capital of Brazil is Brasilia.")

score = output.outputs.score
print(f"Score: {
      score}")
```

代码示例可以在这里找到：[examples/offline_inference/scoring.py](#)

2. 在线服务

我们的 OpenAI 兼容服务器提供了与离线 API 对应的端点：

- Pooling API 类似于 LLM.encode，适用于所有类型的池化模型。
- Embeddings API 类似于 LLM.embed，接受文本和多模态输入，适用于嵌入模型。
- Score API 类似于 LLM.score，适用于跨编码器模型。